Seestar Toolkit User Guide

Development — ENH-003 (unreleased)

Mark Wymer

Document updated: 2026-10-13

This development edition includes the unreleased ENH-001 configuration commands, ENH-002 observing-night date policy and ENH-003 archive light-TIFF option. They are not included in the published v1.1.0 wheel. The released installation instructions and support boundaries remain as described below. No new release number has been assigned.

Contents

- Organise your Seestar captures. Keep your originals safe. Get more from your images.
- What can Seestar Toolkit do?
- What does an archive look like?
- Before you start
  - Supported hardware
  - Supported computer
  - What capture types are supported?
  - A safe way to begin
- Installing Seestar Toolkit
  - 1. Check Homebrew and Python
  - 2. Create a dedicated environment
  - 3. Install Seestar Toolkit
  - Using the Toolkit in a new Terminal session
  - Alternative: install the source distribution
- Your first archive
  - Choose the source and archive folders
  - Preview the archive
  - Inspect the proposed archive
  - Copy the captures
  - Verify the result
  - What you've just done
  - Don't use move yet
- Understanding your archive
  - What is an observation?
  - How is the observing night chosen?
  - What goes inside an observation?
  - Optional individual-light TIFFs
  - Target indexes
  - When the Toolkit is not sure
- Customising your archive
  - Change the hierarchy order
  - Locations
  - Optional configuration
  - Using a different configuration file

  - Which setting wins?
  - Automatic and interactive location handling
  - Customise carefully, then keep it consistent
- Managing configuration
  - Inspect your settings
  - Save or remove an archive preference
  - Inspect a capture's GPS and saved matches
  - Create a saved location
  - Update a saved location
  - Rename or remove a saved location
  - Location names and matching areas
  - Manually edited configuration files and targeted repair
  - Configuration file safety
- Existing archives, collisions and moving original files
  - Re-running an archive operation
  - What is a collision?
  - Preview collisions before doing anything
  - Moving instead of copying
  - A move is not an all-or-nothing transaction
  - Cross-storage operations
- Converting FITS images to TIFF
  - Convert one image
  - What happens during conversion?
  - Batch conversion
  - Batch operations can partially succeed
  - Conversion and archiving are related, but different
  - Where should I put converted TIFFs?
  - Check the result
- Storage and macOS permissions
  - Internal storage
  - External and removable storage
  - Mounted network and NAS storage
  - macOS privacy and permissions
- Privacy
  - Information stored locally
  - Network storage is different from telemetry
- Diagnostics and troubleshooting
  - Confirm that the Toolkit environment is active
  - Which Python does Terminal normally use?
  - Get command help
  - An archive cannot find my captures
  - The archive shows `unknown` as the location
  - The observing-night date looks wrong
  - The Toolkit reports a collision
  - A batch conversion did not convert everything
  - A TIFF looks very dark
  - A copy or move stopped part way through
  - An external drive or NAS is unavailable
  - A configuration file is rejected
  - A saved hierarchy contains an unmatched brace
  - A saved location cannot be changed
  - FITS GPS extraction fails
- Command reference

  - Main command
  - `convert`
  - `convert-batch`
  - `archive`
  - `config`
  - Exit codes
- Uninstalling Seestar Toolkit
  - Deactivate the environment
  - Remove the installed package
  - Remove the whole Toolkit environment
  - Configuration is separate
- Current limitations and getting help
  - Supported means validated
  - v1.1.0 support summary
  - When asking for help
  - Check privacy before sharing diagnostics
  - Keep the originals

Organise your Seestar captures. Keep your originals safe. Get more from your images.

An evening of observing can quickly produce a growing collection of files: individual light frames, stacked images, mosaics and captures from several different targets. After a few nights—or a few months—it can become increasingly difficult to remember what was captured, where it belongs and which files should be kept together.

Seestar Toolkit helps turn that collection into an organised and useful astronomy archive.

The Toolkit examines supported captures, identifies related observations and organises them into a consistent archive based on **target, observing location and observing night**. The archive hierarchy can be customised to suit the way you prefer to organise your images.

Before anything is copied or moved, you can preview the proposed archive and inspect what the Toolkit intends to do. The recommended workflow keeps your original captures safely under your control: **keep a backup, preview first, copy first, and only move files when you are comfortable with the result.**

Seestar Toolkit can also convert supported FITS images to RGB TIFF, individually or in batches, when TIFF is a more convenient format for your image-processing workflow.

What can Seestar Toolkit do?

Organise your observing sessions

Turn supported captures into a predictable archive arranged by **target, location and observing night**. If you prefer a different arrangement, the hierarchy can be customised by changing the order of these components.

The Toolkit uses information from the captures themselves to reconstruct related observations rather than simply treating every directory as an observing session.

Keep related captures together

Individual light frames and Seestar stacked products can be kept together within each reconstructed observation. Your original Seestar stacked images are preserved, and their TIFF companions are generated alongside them—the Toolkit does not create a replacement stack from the individual lights.

This gives each observing night a predictable structure instead of leaving you to manually sort a growing collection of capture files.

Preview before making changes

Archiving doesn't have to mean immediately moving your original files.

The Toolkit can first perform a **dry run**, showing the archive it proposes to create without writing archive outputs. You can inspect the target, location, observing night, observations and proposed file operations before deciding whether to proceed.

For your first archive operations, the recommended approach is simple:

Back up → Preview → Inspect → Copy → Verify

Moving source files is available when you want it, but it should be a deliberate choice after you are familiar with the archive and have suitable backups.

Convert FITS images when TIFF suits your workflow

Seestar Toolkit can convert individual supported FITS images to RGB TIFF, or batch-convert a folder of images in one operation.

Raw Bayer captures are demosaiced during conversion, while supported images that already contain RGB data are handled directly as RGB. The Toolkit produces **linear** images without automatically stretching or altering their brightness, contrast or colour.

Conversion is an additional tool, not a requirement for using the archive. If FITS already suits your processing workflow, you can continue using your original FITS files.

Keep a useful record of your archive

As your archive grows, the Toolkit maintains a human-readable index for each target, providing a convenient record alongside the organised observations.

We'll look at the archive structure and its indexes in more detail later in this guide.

What does an archive look like?

By default, Seestar Toolkit organises captures by:

Target → Location → Observing night → Observation

For example:

```
Astronomy Archive/
│
├── M 31/
│   ├── INDEX.md
│   │
│   └── Home/
│       └── 20260910/
│           └── observation_01/
│               ├── fits_images/
│               │   └── lights/
│               └── seestar_stacked/
│
└── M 57/
    └── ...
```

Here, `M 31` is the target, `Home` is the observing-location label and `20260910` identifies the observing night. Captures belonging to the same reconstructed observation are kept together beneath `observation_01`.

The hierarchy is configurable. For example, if you would rather begin with observing location, the same information could be arranged as:

```
Home/
└── 20260910/
    └── M 31/
        └── observation_01/
            └── ...
```

The Toolkit changes the **order** of the target, location and observing-night components rather than requiring you to adopt one fixed filing system.

Later in this guide we'll explain exactly how observations are reconstructed, how an observing night is determined, what happens to each type of capture and how to customise the hierarchy.

Before you start

There are a few things worth knowing before installing Seestar Toolkit.

Supported hardware

Seestar Toolkit v1.1.0 has been developed and validated with the Seestar S50.

Other Seestar telescope models are not supported by this release unless they are specifically listed as supported. In particular, v1.1.0 does not claim support for the S50 Pro or S30/S30 Pro.

This is a **v1.1.0 support boundary**, rather than a limitation on the future direction of Seestar Toolkit. Throughout this guide we'll therefore generally refer simply to **Seestar**, using a specific model name only where the distinction matters.

Supported computer

Seestar Toolkit v1.1.0 officially supports **Mac computers with Apple silicon**, using a supported version of macOS and Python 3.11, 3.12, 3.13 or 3.14.

Intel Macs, Windows and Linux are not validated as supported platforms for this release. They may work in some circumstances, but they have not been sufficiently tested for v1.1.0.

You don't need to be a Python developer to use the Toolkit. Python provides the environment in which the application runs; once installation is complete, normal use is through commands such as:

```
seestar-toolkit archive ...
seestar-toolkit convert ...
```

The installation section will take you through setting up a dedicated environment so that the Toolkit and its Python packages remain separate from the rest of your Mac.

What capture types are supported?

Version 1.1.0 supports the Seestar FIT/FITS capture structures that have been tested and validated during development, including supported raw Bayer lights, native RGB stacked images and mosaics.

It deliberately does **not** claim that every file produced by every Seestar capture mode or firmware version is supported.

Some types of capture require different processing and are outside v1.1.0. In particular, Solar, Lunar and Planetary video processing is not included in this release. DSLR captures are also outside the v1.1.0 scope.

The supported-input boundaries are summarised later in this guide. For now, the important point is that the Toolkit will report captures it cannot safely classify or process rather than assuming that every FITS file should be treated the same way.

A safe way to begin

If you're primarily interested in organising your existing captures, **don't start by moving them**.

Keep your existing capture collection and backup intact while you learn how the Toolkit interprets it.

A good first experiment is:

1. Choose a small, backed-up Seestar observing session.
2. Ask the Toolkit to preview an archive using **dry-run**.
3. Inspect how it has identified the target, location, observing night and observations.
4. Run the same operation using **copy**, leaving the originals untouched.
5. Examine the resulting archive.
6. Only consider `move` later, if that better suits your workflow.

The archive chapters walk through this process step by step, including what happens when files already exist and how to handle partial operations.

Installing Seestar Toolkit

The recommended installation uses **Homebrew Python** and a dedicated Python virtual environment (usually shortened to **venv**).

A venv gives Seestar Toolkit its own small Python environment. This keeps its packages separate from macOS and from other Python software you may already use.

The basic installation is:

Install or select Python → Create the venv → Activate it → Install Seestar Toolkit → Check the version

You normally do the setup only once. In future Terminal sessions, you simply activate the existing venv before using the Toolkit.

1. Check Homebrew and Python

If you already use Homebrew, check that it is available:

```
/opt/homebrew/bin/brew --version
```

If Homebrew is not installed, follow the official Homebrew installation instructions before continuing.

Seestar Toolkit v1.1.0 supports Python **3.11–3.14**. The examples in this guide use Homebrew Python 3.13.

First, see which Python your Terminal currently finds:

```
/opt/homebrew/bin/brew list --formula | grep '^python'
```

Then check the appropriate supported Homebrew Python explicitly, for example:

```
/opt/homebrew/bin/python3.13 --version
```

The following two commands show which Python your Terminal normally uses:

```
which python3
python3 --version
```

These commands are informational. **Do not modify Apple's system Python.**

To install the Python version used in the examples:

```
/opt/homebrew/bin/brew install python@3.13
/opt/homebrew/bin/python3.13 --version
```

If you already have a supported Homebrew Python 3.11, 3.12, 3.13 or 3.14, you can use that instead.

2. Create a dedicated environment

The recommended location is:

```
~/.venvs/seestar-toolkit/
```

If that directory already exists, inspect it before continuing rather than overwriting it.

Create the environment with your supported Python:

```
/opt/homebrew/bin/python3.13 -m venv ~/.venvs/seestar-toolkit
```

Then activate it:

```
source ~/.venvs/seestar-toolkit/bin/activate
```

Your Terminal prompt may change to show that the environment is active.

Check the selected Python:

```
python --version
```

Activation applies only to the current Terminal session. It does not permanently change your Mac or your normal Python setup.

3. Install Seestar Toolkit

These steps install the **published v1.1.0 release**, not ENH-001. The new configuration commands require an installation containing the development changes; confirm `seestar-toolkit config --help` is available in the environment you intend to use. The package version may still report `1.1.0`, so that number alone does not identify an ENH-001 build.

Download the v1.1.0 release package from the project's GitHub Releases page.

The recommended installation file is the wheel:

```
seestar_toolkit-1.1.0-py3-none-any.whl
```

The Toolkit is **not distributed through PyPI for v1.1.0**, so do not substitute any similarly named package found there.

With the venv active, install the downloaded wheel. Replace the example path with the real location of your file:

```
python -m pip install "/path/to/downloads/seestar_toolkit-1.1.0-py3-none-any.whl"
```

`pip` may download the Toolkit's Python dependencies during installation.

Now check the installation:

```
python -m pip check
seestar-toolkit --version
```

The version command should report:

```
seestar-toolkit 1.1.0
```

Seestar Toolkit can be started in two equivalent ways. The normal and recommended form is the `seestar-toolkit` command used throughout this guide. You can also ask Python to start the Toolkit directly using `python -m seestar_toolkit`.

In Python terminology these are different **entry points**—different ways of starting the same installed application. You do not need to understand Python modules to use the Toolkit; the alternative form is mainly useful for installation checks and troubleshooting.

You can verify the alternative Python entry point:

```
python -m seestar_toolkit --version
```

Both commands should report the same Toolkit version.

Seestar Toolkit is now installed.

Using the Toolkit in a new Terminal session

When you open a new Terminal window later, activate the existing environment before running Toolkit commands:

```
source ~/.venvs/seestar-toolkit/bin/activate
```

You do **not** need to reinstall the Toolkit each time.

When you have finished:

```
deactivate
```

If you prefer not to activate the environment, you can run the Toolkit using its full path:

```
~/.venvs/seestar-toolkit/bin/seestar-toolkit --version
```

For most users, activating the environment is simpler.

Alternative: install the source distribution

The release also includes:

```
seestar_toolkit-1.1.0.tar.gz
```

The wheel is the recommended installation method. If you specifically want to install the source distribution, use a fresh activated venv and run:

```
python -m pip install "/path/to/downloads/seestar_toolkit-1.1.0.tar.gz"
```

`pip` builds the package in an isolated build environment and may need to download build dependencies. After installation, perform the same `pip check` and version checks shown above.

A development checkout, editable installation, `uv` and an installer shell script are not required for normal v1.1.0 use.

Your first archive

If archiving is the reason you installed Seestar Toolkit, this is the best place to start.

For the first run, use a **small observing session that is already backed up**. We are going to preview the proposed archive first, then copy the files. Your source captures will remain in place.

Choose the source and archive folders

You need two locations:

- **Source root** — the Seestar work folder containing the captures you want the Toolkit to examine.
- **Archive root** — a separate absolute path where you want the organised archive to be created.

Archive discovery is deliberately shallow. The Toolkit examines files in the selected source root and files in its immediate child directories; it does not recursively search through an entire disk or folder tree.

A typical source might look like this:

```
work root/                          <- select this as SOURCE_ROOT
  Example Target/
    Stacked_1_Example Target.fit
  Example Target_sub/
    Light_1_Example Target.fit
```

A real-world Seestar work folder may look more like this:

```
My Works/                          <- select this as SOURCE_ROOT
  IC 434/
    Stacked_195_IC 434_10.0s_LP_20260103-225603.fit
  IC 434_sub/
    Light_IC 434_10.0s_LP_20260103-220259_thn.jpg
    Light_IC 434_10.0s_LP_20260103-220259.fit
    Light_IC 434_10.0s_LP_20260103-220259.jpg
```

JPEG and thumbnail files alongside the FITS captures are normal. The archive workflow leaves JPEGs untouched.

Select `work root` itself. If you select a folder above it, the FITS files may be too deep for the Toolkit to discover.

Keep the archive root outside the source tree.

Preview the archive

For this first example we'll provide an observing-location label explicitly. Replace `Home` with the label you want to appear in your archive, and replace both example paths with your actual paths.

Run:

```
seestar-toolkit archive --dry-run --source-action copy \
--collision-policy skip-identical --location "Home" \
--non-interactive "/path/to/work root" "/absolute/path/to/archive"
```

There is a lot in that command, but the important parts for now are:

- `--dry-run` — preview only; don't write the archive outputs.
- `--source-action copy` — when we later perform the operation, retain the source FITS files.
- `--location "Home"` — use this label as the observing location.
- the final two paths are the source and archive roots.

The collision option will make more sense once we discuss existing archives. For this first backed-up example, keep it exactly as shown.

Inspect the proposed archive

Don't rush straight to the next command.

Look at what the dry run reports and ask:

Has it found the captures I expected?

Is the target correct?

Is the observing location correct?

Does the observing night look right?

Have the captures been grouped into sensible observations?

Are the proposed destinations inside the archive root I intended?

Does the operation say `copy`, not `move`?

If something looks wrong, stop here. A dry run is doing its job: it lets you investigate before archive outputs are written.

A successful-looking dry run is still a preview, not a guarantee that a later filesystem operation will succeed. Storage availability and permissions can still change.

Copy the captures

When you are satisfied with the preview, use the same command and remove only **`--dry-run`**:

```
seestar-toolkit archive --source-action copy \
--collision-policy skip-identical --location "Home" \
--non-interactive "/path/to/work root" "/absolute/path/to/archive"
```

The Toolkit can now create the planned archive directories, copy eligible original FITS files, create the Seestar stack TIFF companion where appropriate and maintain the target index.

Because we explicitly selected `copy`, the source FITS files remain in their original location.

JPEGs are left untouched by the archive workflow.

Verify the result

When the command finishes, inspect both sides.

Check that:

1. your original source captures are still present;
2. the expected target/location/night directories exist beneath the archive root;
3. the expected observation directories and files are present;
4. any expected Seestar stack TIFF companion and the target index are where you expect them;
5. the command did not report unresolved observations, collisions or other failures.

Do not delete the original source files simply because the archive command completed.

For now, keep both your backup and your source captures. Use the archive for a while and make sure its organisation suits you.

What you've just done

Without manually sorting the individual capture files, you've asked Seestar Toolkit to:

discover → interpret → group → plan → copy → organise

The default result is arranged as:

```
target/
└── location/
    └── observing-night/
        └── observation_01/
```

The Toolkit may also create `observation_02`, `observation_03` and so on when the captures represent separate observations.

In the next part of the guide we'll look inside that structure properly: how the Toolkit decides what belongs together, how the observing-night date is determined, where lights and stacked products go, what the target index contains, and how the hierarchy can be customised.

Don't use move yet

The Toolkit also supports:

```
--source-action move
```

but there is no need to use it while learning the archive workflow.

Move is intentionally a later topic because files may already have been moved from the source before all the archive processing has finished. If a later step, such as creating a TIFF or updating the index fails, the Toolkit does not automatically undo the work that has already completed.

This is why you should always have a backup, perform a dry run first, and carefully verify the archive after using move.

Once you understand the archive, its collision behaviour and repeated-run behaviour, the guide will show how to preview and perform a move deliberately and safely.

For now:

Back up → Dry-run → Copy → Verify.

Understanding your archive

Once you have created and checked your first archive, the directory structure becomes much more useful if you understand why the Toolkit has arranged it that way.

The Toolkit is not simply copying one Seestar folder into another. It examines supported files and metadata, works out which captures belong together, and reconstructs them as **observations**. Those observations then sit inside the target, location and observing-night structure you saw earlier.

What is an observation?

An observation is a related group of captures that the Toolkit has enough evidence to treat as belonging together. In a straightforward session this might mean a collection of individual light frames plus the Seestar stacked result for the same target.

The grouping is based on capture information rather than just filenames or the folder in which a file happens to be stored. This matters because a directory can contain files from more than one observation, and files with superficially similar names are not automatically safe to combine.

When the evidence supports more than one observation for the same target, location and observing night, the Toolkit keeps them separate:

```
20260910/
├── observation_01/
└── observation_02/
```

Existing observations already present in an archive can influence the next observation number, so do not assume a new run will always create `observation_01`.

How is the observing night chosen?

Astronomy does not fit neatly around midnight. A session that begins late on one calendar day and continues into the early hours of the next is normally still thought of as one observing night.

Seestar Toolkit v1.1.0 handles this with a fixed **12-hour rollover rule**. It takes the selected capture timestamp for the observation, adds 12 hours, and uses the resulting calendar date as the observing-night directory in `YYYYMMDD` form.

For example:

```
Capture time                  Archive observing night
2000-01-01 22:00     ->       20000102
2000-01-02 02:00     ->       20000102
```

Both captures therefore fall under the same night directory. This is a fixed v1.1.0 rule, not a sunset calculation or a time-zone preference.

The capture timestamp comes from the image metadata used by the Toolkit; the modification date of the surrounding folder does not decide the observing night.

What goes inside an observation?

Depending on the captures available, an observation can contain directories such as:

```
observation_01/
├── fits_images/
│   └── lights/
└── seestar_stacked/
```

`fits_images/lights/`

This contains the preserved original FITS light frames belonging to the observation.

`seestar_stacked/`

This contains an original Seestar stacked FITS product when one is available. The Toolkit preserves that stack and creates its required TIFF companion in the same directory. It does **not** recreate a new astronomical stack from the individual light frames.

Normal archive operations preserve individual light FITS files but do not create individual-light TIFF derivatives. Stack-TIFF behaviour remains independent.

Not every observation will necessarily contain every directory. The contents depend on what was present in the source material and what could be safely reconstructed.

Optional individual-light TIFFs

Use `--convert-lights` only when you want the archive operation to create or verify TIFF derivatives for eligible individual lights:

```
seestar-toolkit archive --convert-lights SOURCE_ROOT ARCHIVE_ROOT
```

The archived FITS remains authoritative at `fits_images/lights/`. Successful individual-light derivatives are placed at `tiff_images/lights/`; that directory is created only when at least one TIFF is successfully finalised. This option is for the current invocation only—it is not a saved configuration preference.

The archive generates each candidate from its archived FITS, not from the source work folder. If a TIFF with the primary name already exists, the Toolkit compares the generated candidate with it instead of trusting its filename. An identical TIFF is retained. For a different TIFF, interactive use offers **Keep both**, **Replace existing** or **Skip**; when a `.tiff.duplicate` already exists, the choices are **Replace duplicate** or **Skip**. Skip is the safe default. The Toolkit never makes a numbered duplicate chain. `--non-interactive` safely skips these differing-TIFF collisions.

When the current archive operation encounters identical FITS already archived by that operation, it presents one aggregate prompt, defaulting to Yes. Declining leaves those existing FITS and TIFFs unchanged. With `--non-interactive`, the applicable current-operation set is processed automatically. This is not a recursive archive-maintenance or verification command.

For a preview of this work without any conversion or comparison, use:

```
seestar-toolkit archive --dry-run --convert-lights SOURCE_ROOT ARCHIVE_ROOT
```

Dry-run reports intended derivative work but creates or changes nothing: it does not generate a candidate, compare TIFF bytes or prompt. In a real run, archived FITS is established before derivative conversion. A derivative failure preserves that FITS, leaves existing TIFFs safe, continues with other eligible lights and causes a non-zero result; verified existing TIFFs and safe skips do not by themselves fail the command.

What about the JPEGs?

A normal Seestar work folder can contain JPEGs and thumbnails alongside the FITS files. The archive workflow leaves those JPEG files untouched in the source location; it does not copy or move them into the reconstructed archive.

Target indexes

At the target level the Toolkit maintains a human-readable `INDEX.md` file. This provides a convenient summary alongside the archived observations rather than requiring you to discover everything by browsing directories manually.

For example:

```
M 31/
├── INDEX.md
└── Home/
    └── 20260910/
        └── observation_01/
```

The index is part of the archive, but it is not a substitute for the FITS files or a backup. Because it can contain information derived from your captures—including location and capture information—you should review it before sharing it publicly.

When the Toolkit is not sure

One of the most important archive behaviours is what happens when the Toolkit **cannot** confidently reconstruct an observation.

It does not simply force every readable FITS file into whichever directory looks most plausible. Ambiguous, unresolved or otherwise incompatible material can be reported as a problem instead of being placed automatically.

That can make an archive operation appear more cautious than a manual file-copying script, but the caution is intentional: with original astronomical data, a reported uncertainty is preferable to silently putting captures in the wrong observation.

When a dry run reports a problem, inspect the affected files and the source structure before proceeding. Do not treat a nonzero result as an instruction to bypass the check.

Customising your archive

The default archive hierarchy is designed to be useful without any configuration:

`{target}/{location}/{session_end_date}`

In the guide we normally describe `session_end_date` as the **observing night** because that is what the directory represents to the user.

The default therefore reads naturally as:

Target → Location → Observing night

For example: **M 42/Home/20260915**

You do not have to use that order.

Change the hierarchy order

The three archive components can be reordered to suit the way you prefer to browse your collection. For example:

`{location}/{session_end_date}/{target}`

would give you:

Home/20260915/M 42

This can be useful if you tend to think first in terms of an observing site or a particular night's session rather than an astronomical target.

All three components—`{target}`, `{location}` and `{session_end_date}`—must appear exactly once as separate path components. The option changes their order; v1.1.0 does not provide an arbitrary directory-template language.

For a one-off operation, supply the hierarchy explicitly:

```
seestar-toolkit archive --dry-run --source-action copy \
--hierarchy "{location}/{session_end_date}/{target}" \
--location "Home" --non-interactive \
"/path/to/work root" "/absolute/path/to/archive"
```

As always, preview a changed hierarchy with `--dry-run` before writing archive outputs.

Locations

Location is one of the three archive hierarchy components. You do not necessarily need to specify it every time you create an archive.

If supported Seestar captures contain usable location coordinates, the Toolkit can compare them locally with locations saved in its configuration file. If a capture falls within a saved location's matching radius, the Toolkit can use that site's name automatically. If older saved areas overlap, the nearest containing site is selected. No external geocoder is used.

This is another important reason to start every new archive operation with `--dry-run`.

Look at the proposed destination paths in the dry-run output. If the location has been resolved as expected, you should see its name in the proposed archive structure:

```
M 42/Home/20260915/...
```

If you instead see:

```
M 42/unknown/20260915/...
```

The Toolkit has not resolved the capture to one of your saved locations. Because this is a dry run, nothing has yet been copied or moved, so you can correct the location before proceeding.

If you already know the label you want to use, you can bypass automatic matching by specifying it explicitly:

```
--location "Home"
```

An explicit `--location` takes precedence over saved matching. Run the `--dry-run` again and check the proposed archive paths before performing the actual copy.

Optional configuration

You do **not** need a configuration file to use Seestar Toolkit.

Without one, archive operations use the built-in defaults, including copy, `skip-identical`, and the default target/location/observing-night hierarchy.

Persistent archive preferences and saved observing sites use a configuration file at:

```
~/.config/seestar-toolkit/config.toml
```

This default location is fixed. The Toolkit does not use `XDG_CONFIG_HOME` to select another directory. Use `--config PATH` to select an existing alternative file for an individual command.

The published v1.1.0 release only reads configuration you create manually. **ENH-001 adds `config show`, `config set` and `config unset`** so you can inspect and manage saved entries explicitly. See Managing configuration. Archive-location prompt answers still apply only to the current run.

A simple configuration might look like this:

```toml
[archive]
hierarchy = "{target}/{location}/{session_end_date}"
source_action = "copy"
collision_policy = "skip-identical"

[[locations]]
name = "Home"
latitude = 51.0000
longitude = -1.0000
radius_m = 100.0
```

The double brackets in `[[locations]]` are intentional. They allow repeated saved observing-site entries in the configuration file. Each needs a name, latitude, longitude and radius. You can maintain the file manually, or use ENH-001's location commands to add and check entries for you.

The coordinates above are illustrative only. Replace them with your own observing-site values if you choose to use location matching, and remember that a configuration containing real coordinates is personal data that should be reviewed before sharing.

If you do not want saved coordinate matching, omit the `[[locations]]` entries or remove them individually with `config unset --location NAME`.

Using a different configuration file

You can tell an archive operation to read a particular configuration file:

```
seestar-toolkit archive --dry-run --source-action copy \
--config "/path/to/config.toml" --non-interactive \
"/path/to/work root" "/absolute/path/to/archive"
```

An explicit `--config` path must refer to an existing valid file. If it is missing or malformed, the archive operation reports an error rather than silently falling back to another configuration.

Which setting wins?

When the same setting is supplied in more than one place, the practical rule is:

Command line → Configuration file → Built-in default

So, for example, an explicit `--source-action copy` on the command line overrides a different source action in the configuration file. An explicit `--location` likewise overrides saved location matching.

This is one reason the beginner examples deliberately spell out `copy` and the location rather than relying on hidden defaults: the command itself tells you what the important behaviour will be.

Automatic and interactive location handling

If you omit `--location`, saved-location matching can use capture coordinates as described above. In an interactive Terminal session, the Toolkit can ask you to confirm a saved match or enter a location label manually. A blank manual response means `unknown`. Neither confirmation nor a manually entered label adds or updates saved configuration; use the `config` commands when you want to save a site.

For scripted or unattended use, `--non-interactive` prevents prompting. If no saved location matches, or the capture has no usable coordinates, the location can therefore appear as `unknown` in the proposed archive path.

Treat that `unknown` as useful information rather than something to ignore. Check it during `--dry-run`, resolve the location if necessary, and run the dry-run again before copying or moving files.

Customise carefully, then keep it consistent

The hierarchy and location features are intended to let the archive reflect the way **you** think about your observing collection. They are most useful when you choose a structure and naming convention that you will recognise months or years later.

Before adopting a new hierarchy for important captures, test it with a small backed-up session and `--dry-run`. Once an archive contains many observations, changing your filing convention manually can become a substantial task.

The next chapter deals with the situations that deserve even more care: **existing destinations, repeated archive runs, collisions and moving original files.**

Managing configuration

Development — ENH-001 (unreleased). The commands in this chapter require an installation containing ENH-001. They are not available in the published v1.1.0 wheel. Check `seestar-toolkit config --help` in the environment you intend to use.

Saved configuration settings make archive preferences and observing sites persistent between runs. You can inspect, save and remove them explicitly. An archive command's options still apply to that run only; answers to archive-location prompts are not automatically saved.

The examples below are optional; configuration is not required to use the Toolkit. Choose the examples that suit your needs rather than running them all in sequence. Replace quoted paths and site names with your own. Manual coordinates are illustrative only.

Inspect your settings

Start with:

```
seestar-toolkit config show
```

The command displays the configuration file name and one value for each configurable archive setting. Each value is labelled **saved** or **default**. A saved value overrides the default in this view, even when both values happen to be equal. Saved locations are included.

These settings apply when you run an archive command without overriding them on the command line. For example, `archive --source-action copy` uses copy for that run even if you have saved move as your preference. It does not change the saved value displayed by `config show`.

Command	What it shows
`config show`	Effective archive preferences, their saved/default labels, and saved locations.
`config show --saved`	Saved values in the configuration file, including locations, comments and unfamiliar entries. No missing defaults are added.
`config show --defaults`	Built-in values used when a setting is supplied neither on the command line nor in the configuration file. This command shows those defaults without reading the saved file. There are no default observing sites.

Prefix each command with `seestar-toolkit`.

If the default file does not exist, inspection shows the defaults and creates nothing. If saved values are invalid, inspection identifies the problem and returns an error status. See hierarchy repair for an example of correcting an invalid saved value.

`--saved`, `--defaults` and `--extract` are alternative inspection modes; choose one at a time. `--defaults` cannot be combined with `--config` because built-in defaults are independent of a saved file.

Save or remove an archive preference

These are the archive preferences currently managed by the Toolkit:

Setting	Accepted values	Built-in default
`archive.hierarchy`	`{target}`, `{location}` and `{session_end_date}`, each once as a separate path component, in any order	`{target}/{location}/{session_end_date}`
`archive.source_action`	`copy` or `move`	`copy`

Setting	Accepted values	Built-in default
`archive.collision_policy`	`skip-identical`, `error` or `overwrite`	`skip-identical`
`archive.observation_date_policy`	`start` to identify the observing night by its starting date, or `end` to identify it by its ending date	`end`

For example, save copy as your preferred source action:

```
seestar-toolkit config set archive.source_action copy
```

Or save a different hierarchy order:

```
seestar-toolkit config set archive.hierarchy                                    ↪
'{location}/{session_end_date}/{target}'
```

`set` validates the proposed configuration and reports the change after saving. Replacing an archive preference does not ask another confirmation question. The first successful save can create the absent default file and its directory. Saving a preference does not itself run an archive operation.

To remove a saved preference:

```
seestar-toolkit config unset archive.source_action
```

Later runs use its built-in default unless overridden on the command line. Removing an absent preference changes nothing. Setting an equivalent saved value also avoids rewriting the file; setting an absent preference explicitly to its default does save an entry labelled **saved**.

Saved policy values and `config set` accept different letter cases, but not extra spaces around the value. Newly written policy values use lowercase.

Source and destination paths, `--dry-run`, prompting flags, an explicit archive location and `--config` are choices for an individual command, not saved preferences. There is no `--save-config` or reset-all command.

Inspect a capture's GPS and saved matches

Use one FITS capture from the observing site:

```
seestar-toolkit config show --extract "/path/to/capture.fit"
```

This reads the capture's observing-site coordinates and compares them with saved locations. It **does not save anything**.

The display includes:

- the FITS file and configuration file being checked;
- the extracted latitude and longitude;
- every matching saved site's name, coordinates and radius;
- the distance from the capture coordinates to each matching site's centre.

It distinguishes no saved locations, no match, one match or multiple matches. When several saved areas match the GPS coordinates, matches are listed nearest first. Archive operations still select the nearest match.

If two sites are equally close, the site whose name comes first alphabetically is selected, ignoring capitalisation. For example, `Garden` comes before `Home`. If the names differ only in capitalisation, their original spellings determine a consistent order; for example, `Home` comes before `home`.

GPS is read from the observing-site `SITELAT` and `SITELONG` fields in FITS headers. These are latitude and longitude in degrees. The target's celestial `RA` and `DEC` coordinates are never used as a substitute.

The extractor checks all headers, including repeated site fields. Identical numeric coordinates are acceptable. Conflicting values, incomplete pairs or invalid coordinates produce an error rather than a guessed location. Headers with no site fields are ignored. Extraction reads headers without loading the image data and does not need an image that can be converted to TIFF.

There is no online lookup or coordinate-format conversion. If extraction fails, choose a capture with consistent site coordinates or enter your site's coordinates manually.

Create a saved location

To save the site coordinates from a FITS file as Home:

```
seestar-toolkit config set --location "Home" --extract "/path/to/capture.fit"
```

A **new** location defaults to a matching radius of **100 metres**. You can choose a different radius with `--radius-m`:

```
seestar-toolkit config set --location "Home" \
  --extract "/path/to/capture.fit" --radius-m 150
```

The radius defines the circle within which capture coordinates match the saved site. It is not a distance from the target in the sky.

You can instead enter both coordinates manually:

```
seestar-toolkit config set --location "Example Site" \
  --latitude 10.0 --longitude 20.0 --radius-m 100
```

Replace those illustrative numbers with your site's latitude and longitude. New locations need both coordinates. Do not combine `--extract` with manual latitude or longitude in one command.

A `config set` operation validates the proposed name, coordinates and radius, and checks for conflicts with existing saved locations before saving. Creating a new entry needs no additional confirmation. If the name already identifies an entry, the update rules apply instead.

Update a saved location

Change only the fields you supply. For example:

```
seestar-toolkit config set --location "Home" --radius-m 250
```

This retains its coordinates. You may change latitude or longitude separately for an existing site; the other coordinate and radius remain saved.

To replace both coordinates from a new FITS file:

```
seestar-toolkit config set --location "Home" --extract "/path/to/new-capture.fit"
```

Its existing radius is retained unless you also supply `--radius-m`. The 100 m creation default is not reapplied to an update.

Before a changed existing entry is saved, the Toolkit shows the old and proposed settings and asks:

```
Update config / Skip [Skip]:
```

Enter `Update config` to save, or `Skip` to leave the file untouched. `update` or `u`, and `s` for skip, are also accepted without regard to case. Pressing the Enter key without an answer skips the change.

For an unattended update, authorise the replacement explicitly:

```
seestar-toolkit config set --location "Home" --radius-m 250 --update
```

`--update` requires an existing location and suppresses the question. It never bypasses validation. Without an interactive terminal, a changed existing entry cannot be replaced unless `--update` is supplied.

A valid unchanged edit needs no prompt and does not rewrite the file. Validation still happens first: resubmitting an unchanged older entry can fail if it does not meet the new edit rules.

Rename or remove a saved location

Rename a uniquely identified site:

```
seestar-toolkit config set --location "Home" --rename "Garden"
```

Coordinates and radius are retained. A change in capitalisation alone, such as `home` to `Home`, is allowed. A name used by another site is rejected.

Rename can be combined with coordinate or radius changes:

```
seestar-toolkit config set --location "Home" \
  --rename "Garden" --radius-m 150
```

The complete edit is checked and saved together, with one confirmation question. `--update` can authorise an unattended rename or combined edit.

Renaming changes the saved configuration only. Existing archive directories and indexes keep their current names.

To remove the complete saved entry:

```
seestar-toolkit config unset --location "Home"
```

Removal needs no additional confirmation. If the name is absent, the Toolkit reports that and changes nothing, provided the configuration otherwise passes the required validation. Removing a location does not create a default site or delete archived captures.

Location names and matching areas

Location names are matched without regard to case. Outer spaces are ignored for lookup and removed from newly saved or explicitly renamed names. Internal spaces and your chosen capitalisation are retained. An ordinary coordinate or radius edit preserves an existing name's spelling.

New or edited entries must have:

- a nonblank name, not a standalone `.` or `..`;
- latitude from −90 to +90 degrees;
- longitude from −180 to +180 degrees;
- a finite radius greater than zero metres; decimal values are allowed;
- a name and matching area that do not conflict with another saved entry.

Different names must also remain distinct after the Toolkit cleans them for archive directory use. For example, `A/B` and `a-b` conflict after unsafe characters are replaced and case is ignored.

Saved matching circles must not touch or overlap. If `Home` has a 100 m radius and another site has a 50 m radius, their centres must be more than 150 m apart. A rejected change names the conflicting sites and explains the distance/radius problem. There is no option to force an overlapping save.

These checks apply to the complete proposed location, including rename-only edits. Fix an existing zero radius or overlap as part of the edit if necessary.

Manually edited configuration files and targeted repair

Older manually written configuration files may contain values that do not meet the rules applied during `config set` operations, such as zero-radius sites, duplicate names or overlapping areas. If they satisfy the original required-field, type and range rules, archive operations continue to use them with their existing matching behaviour. Configuration inspection warns about the conflicts.

Unrelated older conflicts do not block a valid preference change or removal. You can repair sites one at a time, but any site you add or edit must pass the new rules. If several old entries have the same trimmed, case-insensitive name, the Toolkit cannot identify one safely: edit the configuration file manually to distinguish them first.

Readable invalid values can be inspected with `config show --saved` and corrected with `set` or `unset`. Another invalid required value may still prevent saving; the proposed document must satisfy its required schema and hierarchy rules, apart from the allowed untouched legacy-location conflicts. An invalid configuration file format requires manual correction.

An omitted radius in an existing configuration-file entry remains an error. The new-location 100 m default does not fill missing values in a hand-written file.

A saved hierarchy with unmatched braces is rejected with a validation message. You can replace it with a valid hierarchy or remove the saved entry to use the built-in default. See troubleshooting.

Observing-night date policy

`archive.observation_date_policy` controls the date used in the existing `{session_end_date}` hierarchy token. Its built-in value is `end`, preserving the established `capture time + 12 hours` rule. Set it to `start` to use `capture time - 12 hours` instead:

```
seestar-toolkit config set archive.observation_date_policy start
```

The fixed boundary is noon: captures from noon through the following morning before noon share one observing night. `start` names it for the first calendar date; `end` names it for the second. Values are case-insensitive when saved and are stored in lowercase. Remove a saved value to restore `end`:

```
seestar-toolkit config unset archive.observation_date_policy
```

There is no per-run archive override. Changing this preference affects later plans only; it does not scan, rename, move, or reorganise existing archives.

Configuration file safety

`--config` PATH selects one existing file instead of the default; the two are not merged. Place it after the `show`, `set` or `unset` verb:

```
seestar-toolkit config show --config "/path/to/existing/config.toml"
seestar-toolkit config set archive.source_action copy \
  --config "/path/to/existing/config.toml"
```

An explicitly selected file must already exist, including for `set`. A missing or unreadable file is an error, not permission to create a replacement or use defaults silently. `show --defaults` remains available without reading a damaged saved file.

When the configuration file is edited:

- Unrelated settings, unfamiliar sections and comments are preserved.
- Formatting around an edited value may change.
- Removing an entry removes its fields and attached inline comments.
- Standalone notes are retained but may move.

If the configuration is a symbolic link—a shortcut to another file—the Toolkit shows the actual destination and updates that target while retaining the link. A broken link or link loop produces an error. Existing file permissions are retained; new default files are readable/writable only by their owner.

The updated file is prepared separately and then replaces the original in one operation. A detected change since reading stops the save and asks you to retry. This protects against accidental edits during a prompt; it is not a multi-user locking system. The Toolkit reports success only after saving completes.

Configuration-save protection is separate from archive execution: an archive copy or move still does not have whole-operation rollback.

Existing archives, collisions and moving original files

Once you have used the archive in copy mode and are comfortable with its structure, the next questions are usually what happens when you run it again, what happens when a file already exists, and whether you should eventually let the Toolkit move the originals.

These are related questions because they all affect existing data. The safest habit remains the same:

Back up → Dry-run → Inspect → Copy or Move → Verify

Re-running an archive operation

You can run the archive command against material that overlaps an archive you have already created. The Toolkit plans the operation against the archive as it exists at that time rather than assuming every run starts with an empty destination.

That means a repeated run may encounter files or observation directories that already exist. Existing observations can also affect numbering: a later operation may create `observation_02` rather than starting again at `observation_01`.

Always inspect the dry-run output before repeating an operation. In particular, check that the proposed target, location, observing night and observation numbering still mean what you expect.

What is a collision?

A collision occurs when the Toolkit wants to write a destination file but something already exists at that path.

For archive operations, the collision policy controls what the Toolkit is allowed to do. Version 1.1.0 provides three policies:

```
skip-identical
error
overwrite
```

`skip-identical`

This is the recommended starting policy and the one used throughout the beginner examples.

If the existing destination is identical to the source that would be written there, the Toolkit can leave the existing file in place and continue. If it is not identical, the operation reports the collision rather than silently replacing different data.

This makes `skip-identical` particularly useful when re-running a previously successful copy operation.

`error`

With `error`, an existing destination is treated as a collision error rather than something the Toolkit should resolve automatically.

Use this when you want an existing destination to stop the affected operation and make the conflict explicit.

`overwrite`

`overwrite` permits an existing destination to be replaced.

That can be useful in a deliberately controlled workflow, but it deserves more care than `skip-identical`. Preview the operation first and make sure you understand exactly which destinations will be affected before allowing existing archive files to be replaced.

Preview collisions before doing anything

A dry run is not only for checking a new hierarchy or an automatically detected location. It is also where you can discover that the destination already contains material that will affect the operation.

For example:

```
seestar-toolkit archive --dry-run --source-action copy \
--collision-policy skip-identical --location "Home" --non-interactive \
"/path/to/work root" "/absolute/path/to/archive"
```

Read the proposed actions rather than treating a successful command as permission to continue automatically. If the output shows an unexpected destination, observation number, unresolved item or collision, investigate it while the operation is still only a preview.

Moving instead of copying

After you have used the archive successfully for a while, you may decide that keeping both the original work directory and the archive copy is unnecessary.

The Toolkit supports:

```
--source-action move
```

A move operation transfers eligible source files into the archive instead of deliberately leaving those source FITS files in their original locations.

This is why `move` is not used in the first-archive tutorial. A mistake in a copy operation still leaves the original captures where you started. A move operation changes that safety equation.

Before using `move`, make sure the captures are independently backed up, run the exact intended operation with `--dry-run`, inspect the proposed structure and collision behaviour, and only then repeat the command without `--dry-run`.

For example:

```
seestar-toolkit archive --source-action move \
--collision-policy skip-identical --location "Home" --non-interactive \
"/path/to/work root" "/absolute/path/to/archive"
```

Verify the completed archive before removing any remaining source material yourself.

A move is not an all-or-nothing transaction

It is important to understand one limitation before using `move`.

The archive workflow is not a whole-operation transaction with automatic rollback. Source removal can occur before every later TIFF or index operation has completed. If a later part of the operation fails, the Toolkit does not promise to put every earlier filesystem change back exactly as it was.

That is why the Guide keeps returning to backups, dry runs and verification.

If an archive operation reports a failure, inspect both the source and archive before running another command. Do not assume either that nothing happened or that everything completed.

Cross-storage operations

Your source and archive do not have to be on the same storage device. The Toolkit has been validated with local storage, removable/external storage and mounted network/NAS storage, including operations between different storage locations.

A mounted network filesystem naturally generates network traffic to that storage. That is different from the Toolkit sending your captures or metadata to an online service.

Storage can disappear, become read-only or suffer an I/O error independently of the Toolkit. This is another reason a successful dry run cannot guarantee that a later filesystem operation will complete.

Converting FITS images to TIFF

Archiving is useful even if you never convert an image. Conversion is a separate tool for occasions when an RGB TIFF is more convenient for your processing workflow.

The Toolkit can convert one supported FITS image at a time or batch-convert the supported images in a directory.

It does not stretch the image for display. The output remains **linear**, preserving the image for later processing rather than automatically changing brightness, contrast or colour to make it look finished.

Convert one image

The basic command is:

```
seestar-toolkit convert INPUT_FITS OUTPUT_TIFF
```

For example:

```
seestar-toolkit convert \
"/path/to/Light_M 42_10.0s_LP_20260915-220000.fit" \
"/path/to/output/M 42.tiff"
```

The input and output filenames are explicit, so you decide where the TIFF is created and what it is called.

The original FITS file is not replaced by the conversion.

What happens during conversion?

Supported raw Bayer captures are demosaiced into RGB during conversion. Supported FITS images that already contain RGB data are handled directly as RGB rather than being demosaiced again.

The resulting TIFF is intended as a faithful linear derivative of the supported FITS input, not as an automatically processed final photograph.

For the normal Seestar integer-image path, the Toolkit produces 16-bit RGB TIFF data. Some supported RGB FITS inputs from other validated processing paths, including supported Siril float data, can result in floating-point TIFF output. For that reason it would be misleading to describe every TIFF produced by the Toolkit as universally 16-bit.

Your FITS file remains the authoritative original capture.

Batch conversion

To convert the supported FITS images in a directory:

```
seestar-toolkit convert-batch INPUT_DIR OUTPUT_DIR
```

For example:

```
seestar-toolkit convert-batch \
"/path/to/fits" \
"/path/to/tiff"
```

Batch conversion is deliberately directory-oriented rather than a recursive search of an entire storage tree. Give it the input directory containing the FITS images you intend to process and a separate output directory for the TIFF files.

Batch operations can partially succeed

A batch contains a series of individual conversions. If one input cannot be converted, do not assume that all earlier successful outputs have been removed or that every later file necessarily failed in the same way.

Read the command output, inspect the TIFF directory and investigate the affected input before deciding whether to repeat the batch.

Conversion and archiving are related, but different

It is useful to keep the two jobs conceptually separate:

Archive reconstructs supported captures into observations and organises them.

Convert creates an RGB TIFF derivative from a supported FITS image.

Convert-batch performs that conversion for a directory of supported inputs.

You do not need to convert your existing FITS collection before archiving it, and you do not need to archive a FITS image before converting it.

The archive workflow creates the TIFF companion for a Seestar-created stacked FITS where appropriate. It preserves individual lights without TIFF derivatives unless the archive invocation explicitly includes `--convert-lights`.

Where should I put converted TIFFs?

The standalone `convert` and `convert-batch` commands do not automatically add their TIFF outputs to a Seestar Toolkit archive.

If you convert your FITS files **before** creating the archive, the archive operation will organise the supported FITS captures but will not discover and move your separately created TIFF files into the archive. If you want to keep those TIFFs with the archived observation, you will need to move them there yourself.

If you convert an image **after** creating the archive, choose an output path yourself. Standalone `convert` and `convert-batch` do not infer or manage an archive's `tiff_images/lights/` directory.

```
observation_01/
├── fits_images/lights/
├── seestar_stacked/
└── chosen-output/      <- your standalone converted TIFFs
```

This is separate from the Seestar stack TIFF companion that the archive workflow creates in `seestar_stacked/`. With the standalone conversion commands, **you choose the TIFF destination explicitly**.

Check the result

After conversion, open or inspect the TIFF using the tools you normally trust for astronomical image work.

Remember that a linear astronomical image can initially look very dark. That does not by itself indicate a failed conversion; the Toolkit has deliberately not applied a display stretch.

Keep the original FITS capture. TIFF conversion gives you an additional image format, not a replacement for the source data.

The remaining chapters cover storage and macOS permissions, privacy, diagnostics and troubleshooting, the command reference, uninstalling the Toolkit and the current limitations of v1.1.0.

Storage and macOS permissions

Seestar Toolkit works with ordinary filesystem paths. Your captures and archive can be on your Mac's internal storage, on removable or external storage, or on a mounted network/NAS filesystem.

Version 1.1.0 has been validated with these storage arrangements, including operations where the source and destination are on different storage devices.

The important requirement is that macOS and your user account allow the Toolkit to read the source and perform the requested operation at the destination.

Internal storage

For files beneath folders you normally use from your account, no special Toolkit configuration is required.

If a command reports that a source cannot be read or a destination cannot be written, check the path first. Also check that the destination exists where you expect it to and that your account has permission to use it.

Do not respond to a permissions problem by changing ownership or permissions across a large part of your Mac. Identify the specific path involved and correct only the underlying access problem.

External and removable storage

An external drive can be used for either the source captures or the archive.

Before starting an archive operation, make sure the drive is mounted and visible in Finder. A path beneath a mounted volume normally begins with:

```
/Volumes/
```

For example:

```
/Volumes/Astronomy/Seestar Archive
```

A dry run can confirm that the Toolkit understands the paths and proposed archive structure, but it cannot guarantee that a device will remain connected or writable during the real operation.

If an external device disconnects or reports an I/O error during a copy or move, inspect both the source and destination before trying again. As discussed earlier, archive execution is not a whole-operation transaction with automatic rollback.

Mounted network and NAS storage

A mounted network share or NAS can also be used as filesystem storage. Mount it in macOS first, then give the Toolkit the mounted path just as you would for another folder.

Network storage naturally involves network traffic between your Mac and that storage. This does **not** mean Seestar Toolkit is uploading your captures to an online service.

A network share can become unavailable during an operation. If that happens, inspect the archive and source before retrying rather than assuming that either the whole operation succeeded or nothing was written.

macOS privacy and permissions

macOS can restrict access to particular folders, removable volumes or network locations. The exact prompts and controls you see can vary with your macOS version and the way your files are stored.

Seestar Toolkit v1.1.0 does **not** require Full Disk Access as a general installation requirement.

If macOS denies access to a location you deliberately selected, review the relevant macOS privacy or filesystem permission for your Terminal application and that location. Grant only the access needed for the files you intend to process.

Privacy

Seestar Toolkit is designed as a local command-line application. Your astronomical images are processed on your Mac using paths you select.

Version 1.1.0 has no telemetry, analytics, automatic update checks or service that sends your FITS images or their metadata to an external server.

The Toolkit also does not use an online geocoder for location matching. Saved observing locations are compared locally with coordinates available to the Toolkit.

Information stored locally

Local processing does not mean that every generated file is free of personal information.

Your FITS files can contain capture metadata. A configuration file can contain the coordinates of your observing sites, and archive indexes or diagnostics can contain information derived from your captures.

Before sharing any of these publicly, review them for information you do not want to publish. In particular, consider FITS headers and capture metadata, your `config.toml` if it contains real observing coordinates, archive `INDEX.md` files, and diagnostic output copied into an issue or support request.

The Toolkit does not automatically remove private metadata simply because a file has been placed in an archive.

Network storage is different from telemetry

If your source or archive is on a mounted NAS or other network filesystem, macOS must send filesystem data across your network in order to read or write that storage.

That traffic is a consequence of the storage location you chose. It is distinct from the Toolkit transmitting captures or metadata to an external application service.

Diagnostics and troubleshooting

When something does not behave as expected, start with the simplest checks before changing files, permissions or configuration.

Confirm that the Toolkit environment is active

Open Terminal and activate the environment:

```
source ~/.venvs/seestar-toolkit/bin/activate
```

Then check:

```
seestar-toolkit --version
python -m seestar_toolkit --version
python --version
```

The two Toolkit commands should report version `1.1.0`, and Python should be one of the supported v1.1.0 versions: 3.11, 3.12, 3.13 or 3.14.

If `seestar-toolkit` is not found but you believe the venv is active, check:

```
which seestar-toolkit
which python
```

With the recommended installation, both should resolve beneath:

```
~/.venvs/seestar-toolkit/
```

You can also check the installed package:

```
python -m pip check
```

Which Python does Terminal normally use?

Outside the activated Toolkit environment:

```
which python3
python3 --version
```

These show which `python3` your shell normally finds. They are not instructions to modify Apple's system Python.

To see installed Homebrew Python formulae:

```
/opt/homebrew/bin/brew list --formula | grep '^python'
```

Get command help

```
seestar-toolkit --help
seestar-toolkit convert --help
seestar-toolkit convert-batch --help
seestar-toolkit archive --help
```

For an ENH-001 installation, also use `seestar-toolkit config --help`.

Use the installed help pages to confirm the exact options accepted by your installed version.

An archive cannot find my captures

Check the source root first. Archive discovery is deliberately shallow: the Toolkit examines the selected source root and its immediate child directories. It does not recursively search an arbitrary folder tree.

If your FITS files are another directory level down, select the appropriate Seestar work folder rather than a directory above it. JPEGs and thumbnails are not archive inputs; their presence beside FITS captures is normal.

The archive shows `unknown` as the location

Use `--dry-run` and inspect the proposed destination. If you see:

```
M 42/unknown/20260915/...
```

the location has not been resolved to the label you expected.

Check the coordinates and matching radius in your configuration. In an ENH-001 build, inspect a capture without saving anything:

```
seestar-toolkit config show --extract "/path/to/capture.fit"
```

This identifies any saved sites containing that capture's coordinates. Alternatively, supply the intended label explicitly for the archive run:

```
--location "Home"
```

Then run the dry-run again before writing the archive.

The observing-night date looks wrong

Remember that v1.1.0 uses the fixed 12-hour rollover rule described earlier. The Toolkit takes the selected capture timestamp, adds 12 hours and uses the resulting calendar date for the observing-night directory.

This is not a sunset calculation and is not based on the modification time of the surrounding directory.

The Toolkit reports a collision

Do not immediately switch to `overwrite`.

First determine what already exists at the proposed destination and why. If you are re-running an operation, `skip-identical` is normally the safer starting policy because an identical existing destination can be left in place while different data is not silently replaced.

Use `--dry-run` before deciding whether another collision policy is appropriate.

A batch conversion did not convert everything

Batch conversion is not recursive and can partially succeed.

Check that the FITS files you intended to convert are directly in the input directory, then inspect the command output and TIFF output directory. Do not assume that a failure affecting one input removed earlier successful TIFFs.

A TIFF looks very dark

The Toolkit creates **linear** image data. It does not automatically apply a display stretch. A linear astronomical TIFF can therefore look very dark when first opened. This does not, by itself, indicate a failed conversion.

A copy or move stopped part way through

Do not immediately repeat the command.

Inspect both the source and archive. Some operations may already have completed before the failure occurred, and the archive workflow does not automatically roll back the whole operation.

Once you understand the state of both sides, use `--dry-run` again before deciding what to do next.

An external drive or NAS is unavailable

Confirm that macOS has mounted the storage and that you can access the intended folder outside the Toolkit.

If the storage disappeared during an archive operation, inspect both source and destination after it becomes available again.

A configuration file is rejected

An explicit `--config` path must name an existing, valid configuration file. If the file is missing or malformed, the Toolkit reports an operational error rather than silently substituting another configuration.

Check the path and configuration-file format. Repeatable saved observing locations use double brackets:

```
[[locations]]
name = "Home"
latitude = 51.0000
longitude = -1.0000
radius_m = 100.0
```

In an ENH-001 build, inspect readable saved entries with:

```
seestar-toolkit config show --saved
```

Correct a recognised invalid preference with `config set` or remove it with `config unset`. The resulting file must pass the required validation before it can be saved. Missing required fields or another invalid value may still need attention. An invalid configuration-file format must be corrected manually.

To inspect built-in defaults without reading a damaged file:

```
seestar-toolkit config show --defaults
```

This does not repair the file or make an archive run ignore it.

A saved hierarchy contains an unmatched brace

The hierarchy must contain `{target}`, `{location}` and `{session_end_date}`, each exactly once as a separate path component. Each placeholder needs both an opening `{` and a closing `}`.

For example, a saved value such as `{target` is an invalid hierarchy, even though the quoted string can be stored in a correctly formatted configuration file. The Toolkit reports the validation error rather than using that value.

Inspect the saved entries with:

```
seestar-toolkit config show --saved
```

Then replace the invalid hierarchy with a complete valid value:

```
seestar-toolkit config set archive.hierarchy                                    ↪
'{target}/{location}/{session_end_date}'
```

Alternatively, remove the saved preference so the built-in default applies:

```
seestar-toolkit config unset archive.hierarchy
```

These are alternative repairs. If another required value is invalid, correct that problem too before the proposed configuration can be saved. Broken configuration-file syntax, such as an unclosed quotation mark, still requires manual correction.

A valid `archive --hierarchy` override applies to that run only; it does not repair the saved value.

A saved location cannot be changed

Read the reported name, coordinate or overlap problem first. A new or edited entry must have a positive radius and a distinct name/matching area. Older conflicts may remain usable for archiving, but do not bypass the edit rules.

If several entries share the requested name after outer spaces and case are ignored, distinguish them manually in the configuration file before using a name-based command. For an intended unattended replacement, use `--update`; it cannot bypass validation and requires an existing entry.

FITS GPS extraction fails

The file needs a consistent observing-site `SITELAT`/`SITELONG` pair. Target `RA`/`DEC` does not identify the site. Choose another suitable capture or enter both site coordinates manually. Conflicting, incomplete or invalid site fields are rejected rather than guessed.

Command reference

This section covers the existing command-line interface plus ENH-001's unreleased `config` commands. The earlier chapters explain the workflows and safety considerations in more detail.

For the exact syntax accepted by the version installed on your Mac, use the relevant `--help` command.

Main command

```
seestar-toolkit [-h] [--version] [--verbose]
                {convert,convert-batch,archive,config} ...
```

The global `--version` and `--verbose` options belong before the subcommand.

The equivalent Python entry point is:

```
python -m seestar_toolkit
```

`convert`

```
seestar-toolkit convert INPUT_FITS OUTPUT_TIFF
```

Example:

```
seestar-toolkit convert \
  "/path/to/input.fit" \
  "/path/to/output.tiff"
```

The source FITS is not replaced.

convert-batch

```
seestar-toolkit convert-batch INPUT_DIR OUTPUT_DIR
```

Example:

```
seestar-toolkit convert-batch \
  "/path/to/fits" \
  "/path/to/tiff"
```

Batch discovery is non-recursive.

archive

```
seestar-toolkit archive [OPTIONS] SOURCE_ROOT ARCHIVE_ROOT
```

The development archive options include:

```
--dry-run
--location LOCATION
--hierarchy HIERARCHY
--source-action {copy,move}
--collision-policy {skip-identical,error,overwrite}
--non-interactive
--convert-lights
--config PATH
```

A safe explicit first-run pattern is:

```
seestar-toolkit archive --dry-run --source-action copy \
  --collision-policy skip-identical --location "Home" \
  --non-interactive "/path/to/work root" "/absolute/path/to/archive"
```

After inspecting the dry run, repeat the same command without `--dry-run` to perform the copy.

Representative archive invocations:

```
seestar-toolkit archive SOURCE_ROOT ARCHIVE_ROOT
seestar-toolkit archive --convert-lights SOURCE_ROOT ARCHIVE_ROOT
seestar-toolkit archive --dry-run --convert-lights SOURCE_ROOT ARCHIVE_ROOT
seestar-toolkit archive --convert-lights --non-interactive SOURCE_ROOT ARCHIVE_ROOT
```

config

ENH-001 development build required. Prefix these forms with `seestar-toolkit`. Square brackets indicate optional arguments and are not typed.

```
config show [--config PATH]
config show --saved [--config PATH]
config show --defaults
config show --extract FILE [--config PATH]
config set PREF VALUE [--config PATH]
config unset PREF [--config PATH]
config set --location NAME [CHANGE FIELDS] [--update] [--config PATH]
config unset --location NAME [--config PATH]
```

`PREF` is `archive.hierarchy`, `archive.source_action`, `archive.collision_policy` or `archive.observation_date_policy`. An explicitly selected PATH must exist.

Location change fields are `--latitude`, `--longitude`, `--extract FILE`, `--radius-m` and `--rename`. At least one change is required. A new site needs both coordinates, supplied manually or from FITS; an existing site permits partial edits. Its radius is retained unless changed explicitly.

Combination	Behaviour
Rename with coordinate/radius edits	Supported for an existing location, as one checked save.
`--update` with an existing-location edit	Supported; skips the question, not validation.
`--extract` with either manual coordinate	Rejected.
Two inspection modes, or `--defaults` `--config PATH`	Rejected.
Scalar and location forms mixed	Rejected.
`set --location NAME` with no change fields	Rejected.
Update/rename/coordinate flags on `unset`	Rejected.

`show` does not save, `set` saves an explicitly requested preference/location, and `unset` removes a saved entry. There is no `config location` subcommand or top-level `config --extract` shorthand.

See Managing configuration for examples and compatibility/repair rules. Command help is available with:

```
seestar-toolkit config --help
seestar-toolkit config show --help
seestar-toolkit config set --help
seestar-toolkit config unset --help
```

Exit codes

```
0    success
1    operational or processing failure
2    command-line usage error
```

An exit code of 1 can represent an invalid input, unreadable or unwritable path, conversion failure, archive/configuration problem or another operational error.

An exit code of 2 normally means the command itself was not formed correctly, such as missing required arguments or an unrecognised option.

For `config`, successful Skip and valid no-op operations also return `0`. Handled configuration, extraction, write and unattended-authorisation failures return 1; unsupported preferences or invalid argument combinations return 2. Legacy-location warnings alone do not make inspection fail. Normal output goes to standard output; warnings and errors go to standard error.

For batch or archive workflows, a nonzero result does not imply that every earlier filesystem action has been rolled back.

Uninstalling Seestar Toolkit

The recommended installation keeps the Toolkit inside its own venv, so removal is straightforward.

Uninstalling the application does **not** mean deleting your captures, generated TIFFs or archive.

Deactivate the environment

If the Toolkit environment is active:

```
deactivate
```

Deactivation only changes the current shell environment. It does not uninstall anything.

Remove the installed package

If you want to uninstall the package while retaining the venv, activate the environment and run:

```
python -m pip uninstall seestar-toolkit
```

After package removal, the `seestar-toolkit` command and `python -m seestar_toolkit` module entry point will no longer be available from that environment.

Remove the whole Toolkit environment

If the venv is dedicated to Seestar Toolkit and you no longer need it, deactivate it and remove:

```
~/.venvs/seestar-toolkit/
```

Deleting the venv removes that Python environment. It does not remove your source captures, TIFF outputs, archive or Toolkit configuration.

Configuration is separate

The default configuration location is:

```
~/.config/seestar-toolkit/config.toml
```

Uninstalling the package or deleting the venv does not delete this file.

If you no longer want the configuration, review it and remove it separately. Generated TIFFs and archive directories likewise remain where you created them unless **you** remove them.

Current limitations and getting help

Version 1.1.0 deliberately has a defined support boundary. That makes it possible to say what has actually been tested and validated rather than implying support for every similar-looking capture or computer.

Supported means validated

Throughout this guide, **supported** means a combination that has been tested and validated for this release.

Seestar Toolkit may work with other telescope models, macOS versions, Python versions, operating systems or computer architectures. Unless specifically listed as supported, those combinations have not been sufficiently tested and validated for v1.1.0, and their behaviour is not guaranteed.

Unsupported therefore does **not** necessarily mean “will not work”. It means that the combination is outside the validated support boundary for this release.

v1.1.0 support summary

The principal v1.1.0 boundaries are:

- **Telescope:** Seestar S50 is the validated Seestar model for this release.
- **Computer:** Apple-silicon Mac.
- **macOS:** an Apple-supported macOS version within the release’s validated platform envelope.
- **Python:** 3.11, 3.12, 3.13 or 3.14.
- **Capture data:** supported Seestar FIT/FITS structures validated for this release, including supported raw Bayer lights, native RGB stacked images and mosaics.

Solar, Lunar and Planetary video processing is not part of v1.1.0. DSLR capture processing is also outside this release.

Other Seestar models, including the S50 Pro and S30/S30 Pro, are not claimed as supported by v1.1.0 unless the release documentation is explicitly updated to say otherwise.

Intel Macs, Windows and Linux are likewise outside the officially supported v1.1.0 platforms.

When asking for help

Before reporting a problem, reproduce it with the safest practical operation. For an archive issue, that usually means `--dry-run` rather than repeating a copy or move.

Useful information can include:

```
seestar-toolkit --version
python --version
```

along with the command you ran, the error or diagnostic output, and a description of what you expected to happen.

For installation or command-line problems, the relevant help output can also be useful:

```
seestar-toolkit --help
seestar-toolkit archive --help
seestar-toolkit convert --help
seestar-toolkit convert-batch --help
```

For an ENH-001 installation, also use `seestar-toolkit config --help`.

Check privacy before sharing diagnostics

Do **not** upload astronomical captures, FITS headers, configuration files or archive indexes without first checking what they contain.

They may reveal observing coordinates, capture times or other information you consider private.

When possible, begin with the command, Toolkit version, Python version and the relevant error text. Provide image files or metadata only when genuinely needed and you are comfortable sharing them.

Keep the originals

Even after you are comfortable with Seestar Toolkit, your original FITS captures remain valuable source data.

The archive and conversion tools are intended to help you organise and use that data, not to make backups unnecessary.

For important captures, keep an independent backup and verify the archive before making decisions about removing source files.

This development guide retains the released v1.1.0 workflows and adds ENH-001 configuration management. Final ENH-001 closure validation remains pending. For a shorter installation and first-use path, see the Seestar Toolkit Quick Start.